

Car Handbook for Teams

Operating, programming and racing the competition car

Version 1.1 · 7 October 2026 · Organising Committee (OC)

Start here. Read chapters 1 to 5 before you power the car on. Keep this handbook open during practice: every command in it can be copied exactly as printed. Text in **this font** is something you type; `<like this>` is a value you replace.

Contents

- 1 Quick start: the first 30 minutes
- 2 Meet your car
- 3 Safety and care
- 4 Connecting to the car
- 5 Daily routine and the car command
- 6 Your code on the car
- 7 From the simulator to the real car
- 8 Camera calibration
- 9 Lane detection and following
- 10 Traffic lights and signs (YOLO on the GPU)
- 11 Obstacles, pedestrians and overtaking
- 12 The race template (crc_team)
- 13 Strategy and planning
- 14 Race day
- 15 Troubleshooting
- A Command cheat sheet
- B Topics and messages
- C Glossary for beginners

1 Quick start: the first 30 minutes

This chapter takes a team from a closed box to a car that drives its first square. Each step is explained in detail later; if something does not work, go to the chapter named in brackets.

1. **Lift the car** so the wheels turn freely (a box under the chassis). Connect a charged battery and switch the car on. The computer needs about 1 minute to start; the drivers start by themselves. (chapter 3)
2. **Connect the USB cable** from the USB port of the Jetson board (next to the power input) to your laptop and open a terminal (Windows: PowerShell; macOS/Linux: Terminal). Log in, password `jetson: ssh jetson@192.168.55.1` (chapter 4)
3. **Check the car: car-check**. Every line should say PASS. A WARN about the sign detector is normal until you install your sign model. (chapter 5)
4. **Join your team Wi-Fi**, then read the car's address: `sudo nmcli dev wifi connect "<network>" password "<password>"` and `hostname -I`. From now on you can log in without the cable: `ssh jetson@<car-address>`. (chapter 4)
5. **Set your ROS domain to your team number** (here team 7): `car domain 7`. (chapter 4)
6. **Copy the example package** into your workspace, start your container and build: `cp -r ~/crc_car/examples/crc_team ~/crc_ws/src/` then `car up` then `car compile`. (chapter 6)
7. **First motion, wheels still lifted**: `car run "ros2 run crc_team drive_square"`. Both wheels turn forward, then the car "turns". Stop with Ctrl+C. (chapter 6)
8. **On the floor**, with 1 × 1 m free in front and to the left: run the same command again. The car drives a 0.5 m square and stops.
9. **Switch off properly**: hold the power button on the board for 1 second (or type `sudo poweroff`), wait until the board lights go out (about 15 s), then switch the battery off. (chapter 3)

Tip. Do the whole quick start together, with one person typing and the others reading the commands aloud. It is the fastest way for every member, including the international members, to learn the car.

2 Meet your car

All competition cars are built the same way and run the same software. The OC has calibrated each car (wheel speed control, camera, LiDAR); your work is the driving software. Take the car you are given and work with it.

2.1 Hardware

Part	What it is	What it gives you
Computer	NVIDIA Jetson with GPU, Ubuntu Linux, Docker	runs ROS 2 and your code; the GPU runs YOLO
Drive	two driven wheels (differential drive), 66 mm wheels, 295 mm apart; motors with encoders	speed control from <code>/cmd_vel</code> , wheel odometry <code>/odom</code>
Driver board	motor driver with IMU and battery sensor, on USB	<code>/imu</code> , <code>/voltage</code>
Camera	CSI camera at the front, about 62° horizontal view	<code>/camera/image_raw</code> : RGB 640 × 480, 30 frames/s
LiDAR	Slamtec RPLIDAR C1, 360°, on top	<code>/scan</code> : 360 rays, 10 scans/s, 0.12 to 3.5 m
Storage	built-in storage for the system, plus a data area at <code>/data</code>	keep datasets, logs and recordings on <code>/data</code>
Battery	3-cell pack, 12.6 V full	about 1 hour of practice; charge below 11.1 V

2.2 Software

Two Docker containers run on the car and talk to each other through ROS 2 topics, exactly like the simulator:

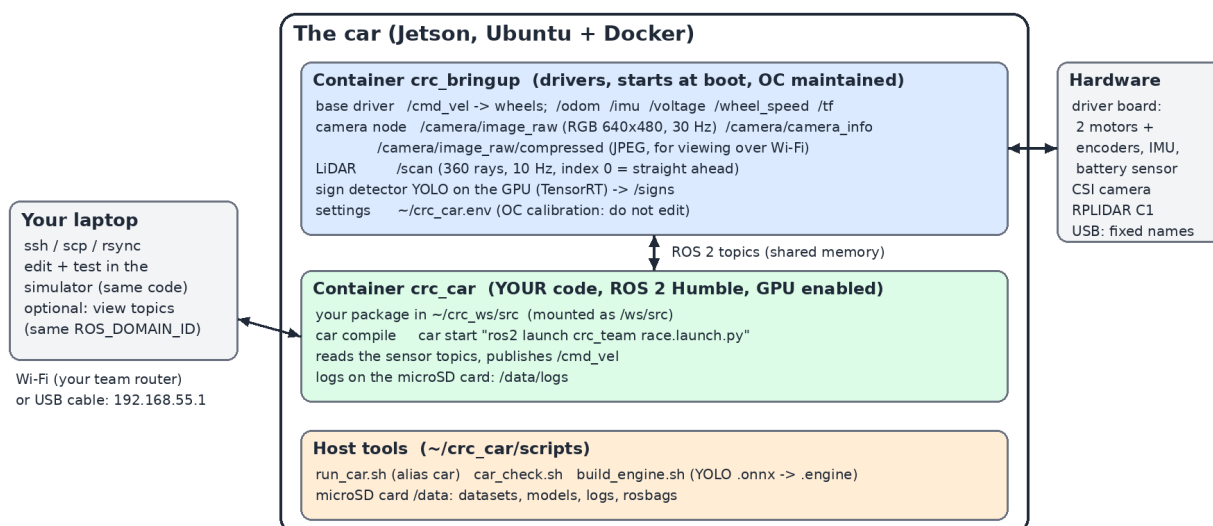


Figure 2.1 Software on the car. You only work in the green container (`crc_car`) and in your folder `~/crc_ws/src`.

- `crc_bringup`** (blue) holds the drivers. It starts at every boot and restarts itself if anything crashes. It offers the same topics as the simulator, so code written for the simulator runs unchanged.
- `crc_car`** (green) is yours: ROS 2 Humble on Ubuntu 22.04 with Python 3.10, OpenCV, NumPy and the GPU libraries. It sees your source folder `~/crc_ws/src` as `/ws/src`.
- The **`car`** command (chapter 5) starts, builds and runs everything; you never need to type a **`docker`** command.

2.3 Conventions

Item	Convention
Frames	x forward, y left, z up (<code>base_footprint</code> is on the floor between the wheels)
<code>/cmd_vel</code>	<code>linear.x</code> in m/s (+ = forward), <code>angular.z</code> in rad/s (+ = turn left, counter-clockwise)
Speed limits	$ \text{linear.x} \leq 1.0 \text{ m/s}$, $ \text{angular.z} \leq 6 \text{ rad/s}$; accelerations are smoothed (0.8 m/s ² up, 1.5 m/s ² down)
Watchdog	the car stops 0.5 s after the last <code>/cmd_vel</code> message: publish at 10 to 20 Hz
<code>/scan</code>	<code>ranges[0]</code> straight ahead, then counter-clockwise (index 90 = left, 270 = right); <code>inf</code> = nothing

Item	Convention
	within 3.5 m
Images	rgb8 (not bgr8): convert before using OpenCV colour functions
/odom	starts at 0,0 when the drivers start; drifts slowly (wheel slip)

3 Safety and care

3.1 Battery

- Charge only with the OC charger, on a non-flammable surface, never unattended. Unplug when full.
- Charge when **car-check** shows the battery as WARN (below 11.1 V). **Below 10.5 V stop at once:** a deeply discharged pack is damaged and may become dangerous.
- A swollen, hot or damaged battery: switch off, disconnect, tell the OC. Never puncture, bend or short-circuit the pack; keep metal tools away from its connector.
- When the car is not in use for more than an hour, unplug the battery.

3.2 Power on and off

- Power on: connect the battery, switch on, wait about 1 minute.
- **Power off:** hold the power button on the board for about 1 second (or type **sudo poweroff**), wait until the board lights go out, then switch the battery off. A short tap does nothing, so a bump during a run cannot stop the car.
- Never cut the power while Linux runs: it can corrupt the car's storage, and the car then has to be re-installed by the OC.
- Never plug or unplug the camera ribbon cable while the car is on.

3.3 Moving car

- First test of any new code: wheels lifted. Only then on the floor.
- Keep fingers, hair and cables away from the wheels and the spinning LiDAR.
- One person is always ready to stop the car: pick it up, or stop the program (**car stop** or Ctrl+C). The car stops by itself 0.5 s after your program stops publishing **/cmd_vel**.
- Start slowly (0.10 to 0.15 m/s) and raise the speed step by step.

3.4 Things you must not do

Do not. change the user, the password or the system settings; run **apt upgrade** or install system packages; edit **~/crc_car.env** (OC calibration) except the **CRC_YOLO_*** lines; edit or delete **~/crc_car**; open the electronics or re-plug USB devices into other ports. If something seems wrong with the car itself, call the OC.

Rules. The car must drive fully autonomously. During a heat nobody touches the car or a keyboard (Rulebook §11). Start your program, close the laptop, step back.

4 Connecting to the car

4.1 USB cable (always works)

Connect the USB port of the Jetson board (next to the power input) to your laptop. The car appears as a network adapter and is always at the same address:

```
ssh jetson@192.168.55.1      # password: jetson
```

- Windows 10/11: use PowerShell or Windows Terminal (ssh is built in). The first time, answer **yes** to the fingerprint question.
- If the address does not answer, wait one more minute (the car is still starting) or try another cable: many USB cables carry power only.

4.2 Your team Wi-Fi

Each team brings its own Wi-Fi router (5 GHz recommended, no internet needed). Connect the car once over the USB cable; it remembers the network:

```
sudo nmcli dev wifi connect "<network>" password "<password>"  
hostname -I                # first number = the car address, e.g. 192.168.0.23
```

Then log in over Wi-Fi from any laptop on the same network:

```
ssh jetson@<car-address>
```

- Give the car a fixed address in your router (DHCP reservation), so it never changes.
- List the networks the car can see: **nmcli dev wifi list**. Forget one: **sudo nmcli con delete "<network>"**.
- The car must not depend on the network during a heat: start your program on the car (chapter 5) so it keeps running when the laptop is closed.

4.3 ROS domain = your team number

All cars and laptops on one network share ROS topics unless they use different **ROS_DOMAIN_ID** values. Use your team number (1 to 12):

```
car domain <team-number>      # sets it for the drivers and your container
```

4.4 Copying files

```
# laptop -> car (run on the laptop)  
scp -r my_folder jetson@<car-address>:~/  
rsync -av --exclude __pycache__ crc_team/ jetson@<car-address>:crc_ws/src/crc_team/  
# car -> laptop  
scp -r jetson@<car-address>:/data/datasets/frames_20261009_101500 .
```

rsync (Linux, macOS, WSL) copies only what changed: ideal for code. On Windows without WSL use **scp -r**. Editors with SSH support (e.g. VS Code Remote-SSH) work, but they use a lot of the car's memory: prefer editing on the laptop and copying.

4.5 The clock

The car has no clock battery, so its clock is wrong after every power-off. It corrects itself from the Internet (HTTPS time) about 20 s after starting and every 15 minutes, as long as its Wi-Fi reaches the Internet. A wrong clock breaks secure downloads and makes RViz on a laptop drop the car's messages (they look too old). **car-check** shows the clock.

Without Internet, set it from your laptop (Linux, macOS or WSL). The time is read when you press Enter, so type the password at once:

```
ssh -t jetson@<car-address> "sudo date -s @$(date +%s)"
```

4.6 ROS 2 between your laptop and the car

ROS 2 programs find each other with multicast messages. Some Wi-Fi networks, including the UEH campus Wi-Fi, block multicast: ROS 2 on your laptop then simply finds no car topics. There is no error message, `ros2 topic list` just shows nothing from the car. Normal (unicast) traffic still works, so `ssh` and `ping` are fine.

The fix is on the laptop only: a small Fast DDS configuration file tells ROS 2 the car's IP address, so the laptop contacts the car directly. Nothing changes on the car. The viewer script writes this file for you:

```
scp jetson@<car-address>:crc_car/scripts/view_car.sh .  
DOMAIN=<team> bash view_car.sh <car-address> profile
```

- **ROS 2 Humble installed on the laptop:** in every terminal, set the two variables the command prints, then restart the ROS daemon (it keeps the old settings):

```
export ROS_DOMAIN_ID=<team>  
export FASTRTPS_DEFAULT_PROFILES_FILE=~/.config/crc/fastdds_peers_<car-address>.xml  
ros2 daemon stop  
ros2 topic list
```

- **No ROS on the laptop** (simulator image only): `view_car.sh` uses the same file for the camera, RViz, rqt or a ROS shell (chapter 8.1): `DOMAIN=<team> bash view_car.sh <car-address> sh`.
- On your own team router multicast usually works and the file is not needed; if topics are missing there too, use the same fix. When the car gets a new IP address, run the `profile` command again.

5 Daily routine and the car command

5.1 The routine

1. Power on, log in (`ssh`).
2. `car-check`: everything PASS? Battery above 11.1 V?
3. `car up`: starts your container (once after every boot).
4. Copy your latest code to the car, `car compile`.
5. Test with `car run "..."` (stops when you press Ctrl+C or the SSH session ends).
6. For real runs: `car start "..."`, then `car logs` to watch, `car stop` to stop.
7. Copy logs and data you want to keep to your laptop.
8. Hold the power button 1 s (or `sudo poweroff`), wait for the lights to go out, then switch off.

5.2 The car command

`car` is a shortcut for `bash ~/crc_car/scripts/run_car.sh` and `car-check` for `bash ~/crc_car/scripts/car_check.sh`. All commands run on the car:

Command	What it does
<code>car up</code>	start your container in the background (GPU enabled, <code>~/crc_ws/src</code> as <code>/ws/src</code>)
<code>car compile</code>	colcon build of your workspace (<code>--symlink-install</code> : Python edits need no rebuild)
<code>car sh</code>	open a shell inside your container with ROS sourced (<code>ros2 topic list, ...</code>)
<code>car run "<cmd>"</code>	run one command attached to your terminal; Ctrl+C stops it
<code>car start "<cmd>"</code>	run your program in the background: keeps running when SSH drops or the laptop is closed; replaces a program that is already running
<code>car logs</code>	follow the output of the background program (Ctrl+C leaves it running)
<code>car stop</code>	stop the background program (like Ctrl+C, then harder)
<code>car status</code>	containers, background program and the list of topics
<code>car frames [s] [n]</code>	save camera JPEGs every s seconds (default 0.5) to <code>/data/datasets</code>
<code>car domain <n></code>	set the ROS domain for drivers and container
<code>car gpu-test</code>	check that the GPU and TensorRT work inside your container
<code>car down</code>	stop and remove your container (your files in <code>~/crc_ws/src</code> stay)
<code>car bringup</code>	restart the drivers (after installing a sign model)
<code>car bringup-logs</code>	driver messages (when a sensor misbehaves)
<code>car-check</code>	self-test of the whole car (about 10 s)
<code>car-check --motion</code>	also turns both wheels for 6 s: lift the car first
<code>car calib-camera</code>	lens calibration with a printed chessboard (chapter 8.3)
<code>car calib-wheels</code>	re-tune the wheel speed controllers, car on the floor (after a crash or a wheel change)

If `car` is not found add the shortcuts once: `echo "alias car='bash ~/crc_car/scripts/run_car.sh'; alias car-check='bash ~/crc_car/scripts/car_check.sh'" >> ~/.bashrc` and log in again.

5.3 The self-test

`car-check` tests the system, storage, network, devices, drivers and every sensor topic, and prints one line per check:

```
== Sensor topics ==
PASS camera          30.0 Hz (want >= 25)
PASS camera image    640x480 rgb8, mean brightness 106/255
PASS lidar /scan     10.0 Hz (want >= 8)
PASS odom            25.0 Hz (want >= 20)
PASS imu gravity      a=(-0.04, -0.01, +9.86) m/s^2, |a|=9.86
PASS battery          12.30 V
```

```
WARN  signs          no /signs (no engine at CRC_YOLO_ENGINE? ...)
Result: 28 PASS, 2 WARN, 0 FAIL
```

PASS = fine, WARN = works but look at it, FAIL = fix before driving. Run it at the start of every session and before every heat; show the output to the OC when you ask for help.

5.4 Logs

`car start` writes everything your program prints to `/data/logs/team_default.log` (the previous run is kept as `team_default.prev.log`). Copy them to your laptop after interesting runs. ROS log files go to `/data/logs/ros`.

6 Your code on the car

6.1 Workspace

Your ROS 2 packages live in `~/crc_ws/src` on the car (`/ws/src` inside the container). Build results go to a separate volume, so the source folder stays clean. Put your solution in **its own package**, not inside `crc_sim`: the same folder then builds in the simulator on your laptop and on the car.

```
~/crc_ws/src/
├── my_team/                # your package (start from crc_team, chapter 12)
│   ├── package.xml
│   ├── setup.py setup.cfg  resource/my_team
│   ├── launch/race.launch.py # ONE command starts everything (Rulebook)
│   ├── config/race_car.yaml config/race_sim.yaml
│   └── my_team/*.py        # your nodes
```

6.2 Build and run

```
car up                # once after boot
car compile            # after adding files or entry points
car run "ros2 run crc_team drive_square" # test, attached
car start "ros2 launch crc_team race.launch.py" # race mode, in the background
car logs              # watch it
car stop
```

Python packages are built with `--symlink-install`: after editing a `.py` file just run again. Rebuild when you add a file, an entry point, a launch or config file.

6.3 A first node, line by line

The smallest useful program: drive forward slowly and stop when the LiDAR sees something closer than 0.3 m straight ahead.

```
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan

class Careful(Node):
    def __init__(self):
        super().__init__('careful')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.create_subscription(LaserScan, '/scan', self.on_scan, qos_profile_sensor_data)
        self.front = float('inf')
        self.create_timer(0.05, self.step) # 20 Hz: the watchdog needs regular
commands

    def on_scan(self, msg):
        ahead = msg.ranges[-10:] + msg.ranges[:11] # -10...+10 degrees
        self.front = min(ahead)

    def step(self):
        cmd = Twist()
        cmd.linear.x = 0.12 if self.front > 0.30 else 0.0
        self.pub.publish(cmd)

def main():
    rclpy.init()
    rclpy.spin(Careful())
```

Add it to `setup.py` under `entry_points('careful = my_team.careful:main')`, `car compile`, then `car run "ros2 run my_team careful"`.

6.4 Reading the camera without cv_bridge

The image arrives as RGB bytes. NumPy turns it into an OpenCV image without copying:

```
import numpy as np, cv2

def on_image(self, msg):
    rgb = np.frombuffer(msg.data, np.uint8).reshape(msg.height, msg.width, 3)
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY) # note RGB, not BGR
    self.latest = rgb # process it in the timer, not here
```

Important. Do the heavy work in a timer on the newest frame, not in the image callback. A callback that is slower than the camera builds up a queue and your car reacts to old images.

6.5 Libraries in your container

Available	Not available (use your laptop)
ROS 2 Humble (rclpy, rclcpp), cv_bridge, image_transport, compressed_images, vision_msgs, tf2, tf_transformations, xacro, teleop_twist_keyboard, rosbag2 Python 3.10, OpenCV 4.5, NumPy 1.21, SciPy, Pillow, PyYAML, pyserial, simple-pid, onnxruntime (CPU) CUDA, cuDNN and TensorRT (trtexec) from the car	PyTorch, Ultralytics (train on a laptop or Google Colab) Gazebo, RViz, rqt (no screen on the car; view from your laptop, chapter 8)

car up creates the container fresh, so anything you **apt install** or **pip install** inside it is lost. Pure-Python helpers can live inside your package. If you really need another library, ask the OC early.

6.6 Performance: the car is not a laptop

- Process small images: crop to the region you need and/or resize to 320×240 .
- Use NumPy/OpenCV operations on whole images; a Python loop over pixels is 100× slower.
- Run your nodes in one process (one launch file with several nodes is fine; avoid many separate Python programs: each costs about 60 to 100 MB of memory).
- Print at most once per second. Logging every frame slows everything down.
- Never subscribe to **/camera/image_raw** from your laptop over Wi-Fi (27 MB/s). Use **/camera/image_raw/compressed** or a small debug image.
- Measure: the race template prints its frame rate and processing time every second.

7 From the simulator to the real car

Your simulator solution is the starting point. Topics and message types are the same; the world is not. Plan time for the differences below.

Topic or item	Simulator	Real car
Track	10 × 6 m board, 350 mm lanes	final track 10 × 4.8 m, 400 mm lanes, 800 mm road
Camera	80° view, 0.10 m high, perfect image	≈62° view, real lens, lighting, glare, motion blur, auto exposure (dark tunnel!)
/odom, /imu	30 Hz / 100 Hz, almost no slip	25 Hz / 25 Hz, wheel slip, drift
/cmd_vel	executed immediately	smoothed (acceleration limits) and stops 0.5 s after the last message
/scan	360 rays, 0.12 to 3.5 m	same format; real surfaces (black objects, glass) may return nothing
Ground truth	/sky_cam, /traffic_lights, /model_states...	none : only the car's own sensors
Signs	only your own detector	/signs from the on-board YOLO detector (chapter 10)
START light	none at the start (junction light only)	real traffic light at START
Package crc_sim	present	not installed: your code must not import it

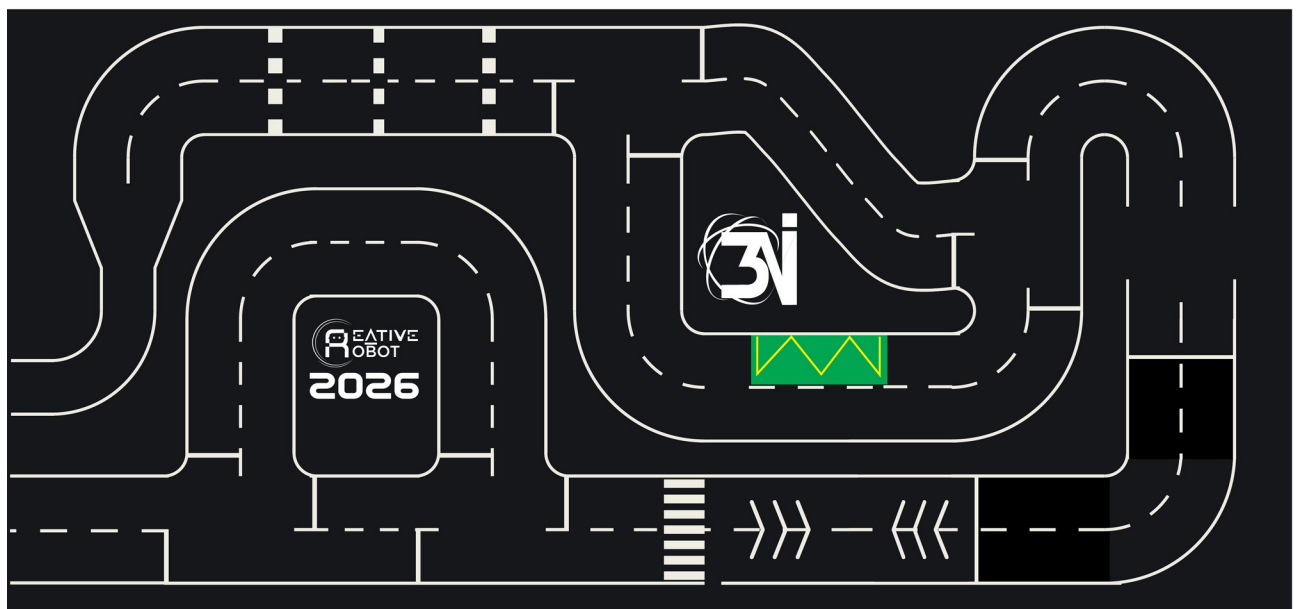


Figure 7.1 The final track (artwork, 10 × 4.8 m, 400 mm lanes). Zone names and local rules: Rulebook §5. Diamonds are placed on the day.

7.1 Porting checklist

1. Move your code into its own package (if it lives in `crc_sim`, copy `starter_node.py` into your package and add an entry point).
2. Replace every number that depends on the world with a parameter: lane width, colours, speeds, distances.
3. Remove ground-truth subscriptions.
4. Build and run it in the simulator again, then copy the same folder to the car.
5. Calibrate the camera (chapter 8) and tune lanes on saved frames (chapter 9) before driving.
6. Drive slowly first, then tune speed.

8 Camera calibration

Lane following needs to know where a pixel is on the floor. That depends on how the camera is mounted, so every car is calibrated once, and again whenever the camera is moved or bumped. Calibration takes about 15 minutes.

8.1 Looking at the camera

- **Single frames:** `car frames 1 5` saves 5 JPEGs (one per second) to `/data/datasets/frames_<date>_<time>/`; copy them to your laptop with `scp -r`.
- **Live, on a laptop with Linux** (or WSL2 with graphics) and the simulator image `crc_sim:humble` from the qualifying round. Copy the viewer script from the car once, then:

```
scp jetson@<car-address>:crc_car/scripts/view_car.sh .
DOMAIN=<team> bash view_car.sh <car-address>      # camera
DOMAIN=<team> bash view_car.sh <car-address> rviz   # LiDAR, odometry, TF and camera
together
bash view_car.sh stop
```

The viewer contacts the car by its address, so it also works on Wi-Fi networks that block ROS discovery (no topics, no error; see 4.6). Other image topics: `bash view_car.sh <car-address> image /lane/debug/compressed` (what the lane detector sees, race template) or `/signs/debug/compressed` (sign boxes). Never view `/camera/image_raw` over Wi-Fi: always the `compressed` topics. If RViz stays empty and complains about old messages, check the car clock (4.5). Saved frames always work.

Keep it clean. Wipe the lens with a dry cloth before each session. A fingerprint blurs the lane lines more than any parameter can fix.

8.2 Bird's-eye calibration (required)

The race template turns the camera image into a top view of the floor in metres ("bird's-eye view"). Four clicks on one photo calibrate it:

1. Put the car on a **straight** lane, centred and pointing along the lane.
2. Lay two strips of tape across the lane, one near and one far, both fully visible in the image. Measure the distance from the **wheel axle** to each strip (e.g. 0.35 m and 0.80 m).
3. Save one frame: `car frames 1 1`, copy it to your laptop.
4. On the laptop (Python 3 with `pip install opencv-python numpy pyyaml`), in the `crc_team/tools` folder: `python3 calib_birdseye.py frame.jpg --near 0.35 --far 0.80`
5. Click where the tape crosses the **centre** of each lane line, in this order: near-left, near-right, far-left, far-right. Press a key.
6. Paste the two printed lines (`birdseye_points`, `birdseye_ground`) into `config/race_car.yaml`.
7. Check `birdseye_preview.jpg`: the lane lines must be straight, vertical, parallel and on the yellow guides.

No screen? Read the four pixel positions with any image viewer and pass them with `--points u1 v1 u2 v2 u3 v3 u4 v4`. Without calibration the template uses the camera height, forward offset and tilt from the config (`cam_height`, `cam_x`, `cam_pitch_deg`), which is only a rough start.

8.3 Lens calibration (recommended)

Real lenses bend straight lines, wide-angle lenses a lot. The car can undistort its own image, so your code sees straight lines like in the simulator. Print `~/crc_car/docs/chessboard_9x6_25mm_A4.pdf` at 100 % (check that one square is 25 mm) and glue it flat on card.

1. `car calib-camera` on the car (`car calib-camera --model pinhole` for a normal lens; by default the tool picks the model for the car's camera setting).

2. Hold the board 0.3 to 1 m in front of the camera and move it slowly: every corner and edge of the image, near and far, tilted. Hold it still for about 1 second in each place. The tool captures 25 views by itself and prints a 3 x 3 map of the image areas covered.
3. It saves `~/crc_calib/camera.yaml` and restarts the drivers. From then on `/camera/image_raw` is undistorted and `/camera/camera_info` holds the matching camera model: your code needs no change.
4. Look at `~/crc_calib/preview.jpg` (left raw, right corrected), then redo 8.2: the image changed.

A reprojection error below 0.5 px is good; above 1 px nothing is saved, try again with sharper views. `car calib-camera --remove` goes back to the raw image.

8.4 Light and colour

- The camera adjusts its exposure automatically. Entering the dark tunnel takes it a moment to adapt: slow down before the tunnel (tunnel sign).
- Stage lights cause glare and reflections on the track. Collect frames at the competition venue and at different times of day, and test your detector on all of them.
- Colour thresholds from the simulator never work unchanged. Tune them on real frames with `python3 tune_hsv.py frame.jpg` (sliders, prints the range).

OC settings. The camera crop, resolution and field of view are set by the OC in `~/crc_car.env`. Do not change them: your calibration would no longer match.

9 Lane detection and following

The race template (`crc_team`, chapter 12) contains a complete lane follower. It runs at camera speed with a few milliseconds per frame and is a good base to improve.



Figure 9.1 Left: camera image. Right: bird's-eye mask with the left line (green), right line (blue), lane centre (red) and the car (yellow). Simulator frame at a junction: the stop lines across the road are ignored.

9.1 How it works

1. **Bird's-eye view:** the gray image is remapped to a top view, 1 px = 6.7 mm, from 0.25 to 0.95 m ahead and 0.6 m to each side (one table lookup, no geometry per frame).
2. **Line mask:** a morphological top-hat keeps thin bright things on the darker road and adapts to the light level (tunnel). Rows that look like a zebra crossing, lines across the road and blobs wider than a line are ignored.
3. **Sliding windows:** starting where each line was in the last frame (or $\pm$ half a lane), windows climb the image and follow the line, predicting where a curving line goes next; dashed lines may skip windows. Two lines that are not one lane apart near AND far (e.g. a stop line taken for a lane line) are rejected.
4. **Fit:** a curve $y(x)$ in metres for each line found; if only one line is visible, the centre is half a lane width away from it, measured square to the line (this matters in tight bends).
5. **Steering (pure pursuit):** aim at the lane centre point (x, y) about **lookahead** metres ahead. The circle through the car and that point has curvature $2y/(x^2+y^2)$; angular speed = speed $\times$ curvature; speed drops in bends.
6. **Lost lines:** keep the last steering for 0.5 s, then drive straight slowly for **blind_distance** (zebra, junction, ramp), then stop.
7. **Obstacles:** LiDAR points in a corridor that bends with the current steering arc (so walls in bends are not obstacles) stop the car.

9.2 Parameters that matter

Parameter	Default (car)	Effect
<code>lane_width</code>	0.425	distance between line centres (400 mm lane + 25 mm line)
<code>min_contrast</code>	25	lower = sees fainter lines (tunnel), but more noise
<code>be_x_min, be_x_max</code>	0.25, 0.95	how far the detector looks; must be inside the camera view
<code>lookahead</code>	0.45	larger = smoother but cuts corners; smaller = follows tightly but wobbles
<code>max_speed, min_speed</code>	0.25, 0.10	speed on straights / lowest speed in bends
<code>curve_slowdown</code>	1.0	how much the car slows down in bends
<code>blind_distance</code>	1.0	how far to go straight when no line is visible

9.3 Tuning procedure

1. Drive the car around the whole track slowly by hand (teleop, below) while saving frames: `car frames 0.3`.
2. Run the detector on all of them, on the laptop or on the car: `python3 lane_debug.py 'frames/*.jpg' --config ../config/race_car.yaml`. Look at the annotated images in `frames/lane_debug/` and fix settings until the lane is found in almost every frame.
3. Drive with `max_speed: 0.12`. Watch `car logs: lines=LR` both lines, `.R` right only, `..` none.
4. Raise the speed in steps of 0.03 m/s. Wobbling on straights: increase `lookahead`. Cutting corners or touching the inner line: decrease `lookahead` or increase `curve_slowdown`.
5. Write down every change and its result. Change one thing at a time.

Manual driving for data collection (second SSH window, keys i/j/l/k/,):

```
car run "ros2 run teleop_twist_keyboard teleop_twist_keyboard"
```

9.4 Zone tips

Zone	What to expect	Idea
MISSING LINE	no centre line	follow the right line only (<code>prefer = 'right'</code>) with the half-lane offset
DARK TUNNEL	low light, slow exposure	slow down at the tunnel sign; lower <code>min_contrast</code>
BUS	S-curve with a reserved lane	never cross into the BUS lane (-10): slower, shorter lookahead
PEDESTRIAN	zebra crossing	zebra rows are ignored; stop if someone is on it (chapter 11)
RAMP	up and down, chevrons	chevrons are ignored; no stopping on the ramp
Junctions	lines in all directions	follow the line on the side you want to turn to (<code>prefer = 'left'/'right'</code>) for a few seconds
HIGHWAY	long straight, dashed centre line	faster (<code>highway_speed</code>), overtaking allowed

10 Traffic lights and signs (YOLO on the GPU)

Every car runs a YOLO object detector on its GPU (TensorRT, FP16) inside the drivers. You only provide the trained model; the car publishes the detections on `/signs` at up to 15 Hz. Your code decides what to do with them.

10.1 The `/signs` message

`vision_msgs/Detection2DArray`, one entry per detected object, coordinates in pixels of the 640 × 480 camera image:

```
from vision_msgs.msg import Detection2DArray

def on_signs(self, msg):
    for det in msg.detections:
        name = det.results[0].hypothesis.class_id      # e.g. 'stop' (from your labels file)
        score = det.results[0].hypothesis.score        # 0..1
        cx, cy = det.bbox.center.position.x, det.bbox.center.position.y
        w, h = det.bbox.size_x, det.bbox.size_y        # box size in pixels
        if name == 'stop' and h > 40:                 # tall box = close sign
            ...
```

Distance from the box height: $d \approx f \cdot H / h$, with $f \approx 533$ px (62° camera, 640 px wide), H the real height of the sign plate in metres and h the box height in pixels. Example: a 70 mm plate seen 75 px tall is about 0.5 m away. Check this on your car with a tape measure. The image with boxes is on `/signs/debug/compressed`.

10.2 Train your own sign model

1. **Classes.** Decide the class names first; they must match your code and config. Suggested: `stop`, `pedestrian`, `bus`, `tunnel`, `hump`, `uneven_road`, `hw_enter`, `hw_exit`, `turn_left`, `turn_right`, `go_straight`, `red_light`, `yellow_light`, `green_light`. Add the direction signs once the OC publishes their design (Rulebook §5.2).
2. **Photos from the car itself**, on the real track: drive slowly by hand with `car frames 0.3`. Aim for 150 to 300 images per class: near and far, from the side, in the tunnel, with glare, partly hidden. Add images with no sign at all.
3. **Label** the boxes (Roboflow, CVAT or labellmg; export in YOLO format). Split 80 % train / 20 % validation.
4. **Train** on a laptop with a GPU or on Google Colab (Ultralytics):

```
pip install ultralytics
yolo detect train data=signs.yaml model=yolov8n.pt imgsz=416 epochs=100 batch=32
yolo export model=runs/detect/train/weights/best.pt format=onnx imgsz=416 opset=12
simplify=True
```

5. **Labels file:** the class names in training order, one per line (`labels.txt`).
6. **Copy to the car and build the engine** (on the car itself, not in the container; takes several minutes):

```
scp best.onnx labels.txt jetson@<car-address>:~/          # on the laptop
bash ~/crc_car/scripts/build_engine.sh ~/best.onnx ~/labels.txt signs # on the car
car bringup                                              # restart the drivers
car run "ros2 topic echo /signs"                        # see detections
```

- YOLOv5, YOLOv8 and YOLO11 models all work; use the smallest size, "n" (e.g. `yolov8n.pt`). Inputs of 320 or 416 pixels are much faster than 640 and good enough for signs at 1 m.
- The engine is built for the car it is built on: build it on your own car.
- Confidence threshold and maximum rate: `CRC_YOLO_CONF` (default 0.4) and `CRC_YOLO_RATE` (default 15) in `~/crc_car.env`, then `car bringup`. These are the only lines of that file you may change.
- Keep your dataset and the `.onnx` on `/data/models` and on a laptop: they are your most valuable files.

10.3 Traffic lights

Two ways, best used together:

- **Colour detector** (`crc_team/light.py`): HSV thresholds in the upper part of the image, round blobs only. Fast, no training. Fooled by other red objects (STOP signs!), so the template uses it only at START, where the car stands still facing the light.
- **YOLO classes** `red_light`, `yellow_light`, `green_light`: robust anywhere, needs training images of the real lights (lit and unlit lamps, near and far).

Rule reminder. Running a red light costs –30 points, the same as 30 seconds. When in doubt, stop.

11 Obstacles, pedestrians and overtaking

11.1 Reading the LiDAR

`/scan` has 360 rays, one per degree, counter-clockwise from straight ahead. The template turns them into points in the car frame and checks a corridor in front of the car:

```
r = np.asarray(scan.ranges, np.float32)
a = scan.angle_min + np.arange(r.size) * scan.angle_increment
ok = np.isfinite(r)
x, y = r[ok] * np.cos(a[ok]), r[ok] * np.sin(a[ok])      # metres, x forward, y left
in_lane = (x > 0.12) & (x < 0.35) & (np.abs(y) < 0.15)  # ignore the car's own body
blocked = in_lane.sum() >= 2                          # 2 rays: ignore single noise
points
```

11.2 Pedestrian crossing

The pedestrian is a figure that crosses the zebra. Failing to yield costs -30. The template stops whenever something is in its corridor closer than `stop_distance` and drives on when the corridor is clear. Add the pedestrian sign as a "slow" sign so the car arrives slowly.

11.3 Overtaking

- Allowed on the HIGHWAY (between the HW signs) and where the centre line is dashed; crossing a solid centre line to overtake costs -15.
- The template overtakes only after it has seen the highway-entry sign and the obstacle stayed in front for `overtake_wait` seconds: it shifts its target one lane to the left for `overtake_length` metres, then returns when the right lane is clear.
- In two-car heats the car in front has priority, and the overtaking car is responsible for any collision (Rulebook §9). A clean slower heat beats a collision.

12 The race template (crc_team)

`~/crc_car/examples/crc_team` is a complete, working starting point that runs both in the simulator and on the car. Copy it, rename it if you like, and improve it.

File	Content
<code>crc_team/race_node.py</code>	the driver: state machine, lane following, signs, obstacles
<code>crc_team/lane.py</code>	lane detector (OpenCV, no ROS: testable offline)
<code>crc_team/birdseye.py</code>	camera image → top view in metres; geometry or 4-point calibration
<code>crc_team/light.py</code>	traffic light colour detector
<code>crc_team/config.py</code>	all settings with their defaults
<code>crc_team/drive_square.py</code>	first program: drive a square with /odom
<code>config/race_car.yaml</code>	settings for the real car (edit this one)
<code>config/race_sim.yaml</code>	settings for the simulator
<code>launch/race.launch.py</code>	one command: <code>ros2 launch crc_team race.launch.py [config:=sim]</code>
<code>tools/</code>	<code>calib_birdseye.py</code> , <code>calib_intrinsics.py</code> , <code>lane_debug.py</code> , <code>tune_hsv.py</code>

12.1 Running it

```
# simulator (laptop, Student Pack container: copy crc_team into src/ and compile)
ros2 launch crc_team race.launch.py config:=sim
# car
car run "ros2 launch crc_team race.launch.py"          # test
car start "ros2 launch crc_team race.launch.py"        # race
```

It prints one status line per second:

```
DRIVE      lines=LR offset=+0.012 m frames=20/s lane=2.4 ms highway=False dist=12.3 m
```

12.2 The state machine

State	Behaviour	Leaves when
WAIT_START	stands still, watches the START light	green seen in 3 frames in a row (<code>wait_for_green: false</code> skips it)
DRIVE	lane following; stops for obstacles; reacts to signs	obstacle on the highway → OVERTAKE
OVERTAKE	lane following one lane to the left	driven <code>overtake_length</code> and the right lane is clear
(stop sign)	stands still for <code>stop_time</code>	time is up
(red light)	stands still while a red light is seen	green seen, or no red for <code>red_clear_time</code>

12.3 Adding a reaction to a new sign

Signs are mapped to actions by class name in the config:

```
stop_classes: [stop]                # stop for stop_time seconds
slow_classes: [pedestrian, tunnel, hump, bus, uneven_road] # slow_factor for slow_time
left_classes: [turn_left]           # follow the left line for branch_time seconds
hw_enter_classes: [hw_enter]        # highway: faster, overtaking allowed
```

For a new behaviour, add a branch in `on_signs()` and, if it needs its own driving, a new state in `step()`. Keep every number in the config file.

12.4 Known limits (your job)

- Junctions: the template does not know the route. Use the direction signs (afternoon heats) and your own route plan (morning solo run).
- Diamonds are not detected: the OC announces their layout on the day (Rulebook §6.2).

- The traffic light in the middle of the track needs the YOLO light classes.
- Two-car heats: no tracking of the other car beyond "something is in front of me".
- Speeds are deliberately low. Make it clean first, then fast.
- Where all lines disappear for longer than **blind_distance** (some junctions, wide merges), the template stops and waits: add your own junction handling there.
- Tested in the simulator: straights, junction crossings, zebra, ramp, the tunnel and its bend, the right-hand loop. Real-track behaviour depends on your calibration and tuning.

13 Strategy and planning

13.1 What the points tell you

TOTAL = diamonds $\times$ 10 + 50 for a legal finish + seconds left (out of 300) – penalties. One second is one point (Rulebook §6). Consequences:

- **Finishing is everything.** A DNF gets no finish points, no time points and only 70 % of its diamonds. Reliability first, speed second.
- **Penalties are expensive seconds.** A line touch (–5) is 5 s; a red light, a pedestrian or driving against traffic (–30) is half a minute; a manual reposition is –15 each.
- **Slower can win.** Driving 20 % slower costs maybe 20 s and can save several –5 line touches and a –15 reposition.
- **Routes are balanced:** a longer route carries about as many diamond points as the time it costs. Choose the route your car drives cleanly. Never chase a diamond off the road (voided).
- Do not stand still for more than 10 s except at a red light or for a pedestrian (–10).

13.2 Team roles (6 members)

Role	Owns
Lead / integration	race node, config, launch file, decisions, the final version
Lanes (2)	camera calibration, lane detector, speed tuning per zone
Signs	dataset, labelling, training, engine, sign reactions
Test & operations	battery, car-check, logs, timed runs, the test notebook, backups
Rules & route	Rulebook, route choice, zone rules, race-day checklist

Pair every international member with a local member on a concrete task from the first hour. Everyone must be able to start and stop the car.

13.3 Plan for the three days

When	Goal
Before Day 1	your package runs in the simulator and is in its own package; laptops have ssh, rsync/scp, Python + OpenCV; a labelling account; this handbook read
Day 1 morning	car handover; quick start (chapter 1); car-check; drive_square; set Wi-Fi and domain
Day 1 afternoon	camera calibration; save frames of the whole track and of every sign and light; lane follower at 0.12 m/s around the track
Day 1 evening	label and train the sign model; export ONNX
Day 2 morning	build the engine; sign reactions; START light; pedestrian; speed tuning
Day 2 afternoon	full timed runs of your chosen route; fix the worst fault first; 16:30 running-order draw
Day 2 evening	freeze the code: copy it to two laptops; write the race checklist
Day 3	08:15 calibration run with the frozen version; only config changes after that

13.4 Working habits that win

- One change at a time, one test notebook: date, version, change, result, time, faults.
- Keep a "known good" version (copy the package folder with the date) before every experiment, and go back to it without hesitation.
- Test full laps, not only the zone you are working on: new code breaks old zones.
- Watch the battery: a weak battery changes the car's speed and your tuning.
- Use the logs: **car logs** during the run, **/data/logs** after it.

14 Race day

14.1 Before each heat

1. Fully charged battery (above 12.0 V), spare battery charged.
2. **car-check**: no FAIL. Lens clean.
3. **car up** done; your final version compiled; the correct config.
4. Set the clock if needed (chapter 4.5) and test one start in the practice area.

14.2 At the start

1. Place the car on START as the referee asks.
2. **car start** "`ros2 launch <your_package> race.launch.py`" and check with **car logs** that it waits for the green light (WAIT_START).
3. Close the laptop, step back, hands off (Rulebook §11).
4. The car leaves on the green light. A manual reposition is only done when the referee calls it (-15).

14.3 After the heat

- **car stop**, copy `/data/logs/team_default.log` to a laptop and note what happened.
- Battery swap: hold the power button 1 s (or **sudo poweroff**), wait, switch off, swap, power on, **car-check**, **car up**.
- Sign the scoresheet; appeals within 5 minutes (Rulebook §11).

15 Troubleshooting

Symptom	What to do
ssh: connection refused or timeout	wait 1 minute after power-on; use the USB cable (192.168.55.1); check the car address with <code>hostname -I</code> over USB
<code>car: command not found</code>	add the shortcuts (chapter 5.2) or use <code>bash ~/crc_car/scripts/run_car.sh</code>
"Team container is not running"	<code>car up</code>
<code>ros2 topic list</code> shows few topics	wrong domain: <code>car domain <n></code> and <code>car up</code> ; drivers restarting: <code>car bringup-logs</code>
the car does not move	is your node publishing? <code>car run "ros2 topic hz /cmd_vel"</code> ; battery; <code>car-check --motion</code> with wheels lifted
the car drifts or wobbles	camera calibration (chapter 8); lookahead and speed (chapter 9)
the lane is lost in one zone	save frames there, run <code>lane_debug.py</code> , adjust
no /signs	no engine yet, or wrong name: <code>car bringup-logs</code> ; the file must be <code>~/crc_engines/signs.engine</code> (or set <code>CRC_YOLO_ENGINE</code>)
permission denied in <code>~/crc_ws/src</code>	files created by the container belong to root: <code>sudo chown -R jetson: ~/crc_ws/src</code>
colcon: "can't copy ... doesn't exist"	<code>car down</code> , <code>docker volume rm crc_build_default</code> , <code>car up</code> , <code>car compile</code>
secure download / apt errors, RViz drops "old" messages	the clock is wrong (chapter 4.5)
the laptop sees no car topics, no error (<code>ssh</code> works)	the Wi-Fi blocks multicast: Fast DDS profile with the car address (chapter 4.6)
camera image black or frozen	<code>car-check</code> ; <code>sudo systemctl restart crc-camera</code> ; if it persists, power off and call the OC (ribbon cable)
memory full / very slow	<code>free -h</code> ; close VS Code Remote; stop extra programs; datasets belong on /data
anything else	run <code>car-check</code> , copy its output and the last lines of <code>car logs</code> , and call the OC

A Command cheat sheet

Task	Command
log in (cable / Wi-Fi)	<code>ssh jetson@192.168.55.1 · ssh jetson@<car-address></code>
join Wi-Fi	<code>sudo nmcli dev wifi connect "<network>" password "<password>"</code>
car address	<code>hostname -I</code>
team domain	<code>car domain <n></code>
self-test	<code>car-check · car-check --motion (wheels lifted)</code>
start container	<code>car up</code>
build	<code>car compile</code>
shell with ROS	<code>car sh</code>
test a program	<code>car run "ros2 run <pkg> <node>"</code>
race mode	<code>car start "ros2 launch <pkg> race.launch.py" · car logs · car stop</code>
camera frames	<code>car frames 0.5 (Ctrl+C to stop) · car frames 1 10</code>
manual driving	<code>car run "ros2 run teleop_twist_keyboard teleop_twist_keyboard"</code>
record topics	<code>car run "ros2 bag record -o /data/bags/run1 /scan /odom /camera/image_raw/compressed"</code>
sign engine	<code>bash ~/crc_car/scripts/build_engine.sh best.onnx labels.txt signs</code> then car bringup
topic rate / content	<code>car run "ros2 topic hz /scan" · car run "ros2 topic echo /odom --once"</code>
copy code to the car	<code>rsync -av --exclude __pycache__ my_team/ jetson@<car-address>:crc_ws/src/my_team/</code>
set the clock (laptop)	<code>ssh -t jetson@<car-address> "sudo date -s @"\$(date +%s)"</code>
view the car (laptop)	<code>bash view_car.sh <car-address> · ... rviz (chapter 8.1)</code>
power off	hold the power button 1 s, or <code>sudo poweroff</code>

B Topics and messages

Topic	Type	Rate	Notes
/cmd_vel	geometry_msgs/Twist	yours, 10-20 Hz	input; stops after 0.5 s without messages
/camera/image_raw	sensor_msgs/Image	30 Hz	rgb8, 640 × 480
/camera/image_raw/compressed	sensor_msgs/CompressedImage	30 Hz	JPEG, for viewing
/camera/camera_info	sensor_msgs/CameraInfo	30 Hz	ideal pinhole model
/scan	sensor_msgs/LaserScan	10 Hz	360 rays, 0 = ahead, CCW, 0.12-3.5 m, inf = none
/odom	nav_msgs/Odometry	25 Hz	odom → base_footprint, also on /tf
/imu	sensor_msgs/Imu	25 Hz	gravity on +z ≈ 9.8 m/s ²
/signs	vision_msgs/Detection2DArray	≤ 15 Hz	your YOLO model; class_id = label
/signs/debug/compressed	sensor_msgs/CompressedImage	on demand	image with boxes
/voltage	std_msgs/Float32	25 Hz	battery volts
/wheel_speed	std_msgs/Float32MultiArray	25 Hz	[left, right] wheel speed in rad/s
/tf, /tf_static	tf2_msgs/TFMessage		base_footprint, base_link, base_scan, imu_link, camera_link, camera_rgb_optical_frame

C Glossary for beginners

Word	Meaning
SSH	secure remote terminal: you type on your laptop, commands run on the car
Container (Docker)	a sealed software box with its own Linux and libraries; the car runs two
ROS 2 node	one program that exchanges messages with others
Topic	a named channel of messages, e.g. /scan; nodes publish to it or subscribe to it
ROS_DOMAIN_ID	a number that separates groups of ROS programs on one network
Package	a folder with ROS code and a package.xml; built with colcon
Launch file	starts several nodes with their settings in one command
Odometry	position estimated from wheel rotation; drifts over time
IMU	motion sensor: acceleration and rotation speed
LiDAR	spinning laser that measures distances all around
Bird's-eye view	the camera image transformed into a top view of the floor
YOLO	a fast neural network that finds objects and draws boxes around them
TensorRT engine	a YOLO model compiled for the car's GPU (.engine file)
Pure pursuit	steering rule: aim at a point on the path ahead

Questions about the car: ask the OC technical desk. Rules: the Rulebook and its updates take precedence over this handbook.